

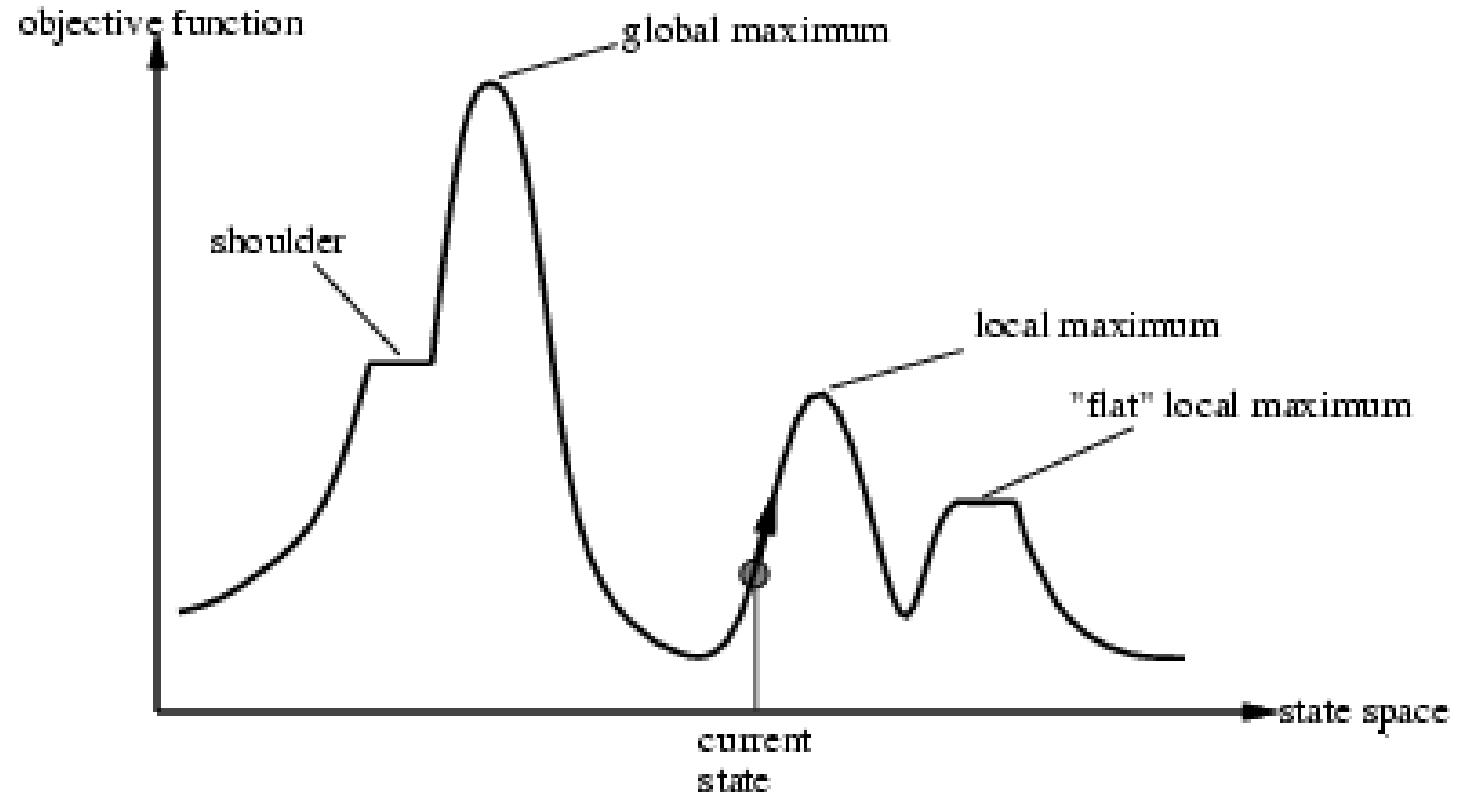
# Beyond Classical Search

# Local search algorithms

- In many optimization problems, the **path** to the goal is irrelevant; the goal state itself is the solution
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n-queens
- In such cases, we can use **local search algorithms**
- Keep a single "current" state, try to improve it

When elevation corresponds to **objective function**, the aim is to find the **highest peak – global maximum**.

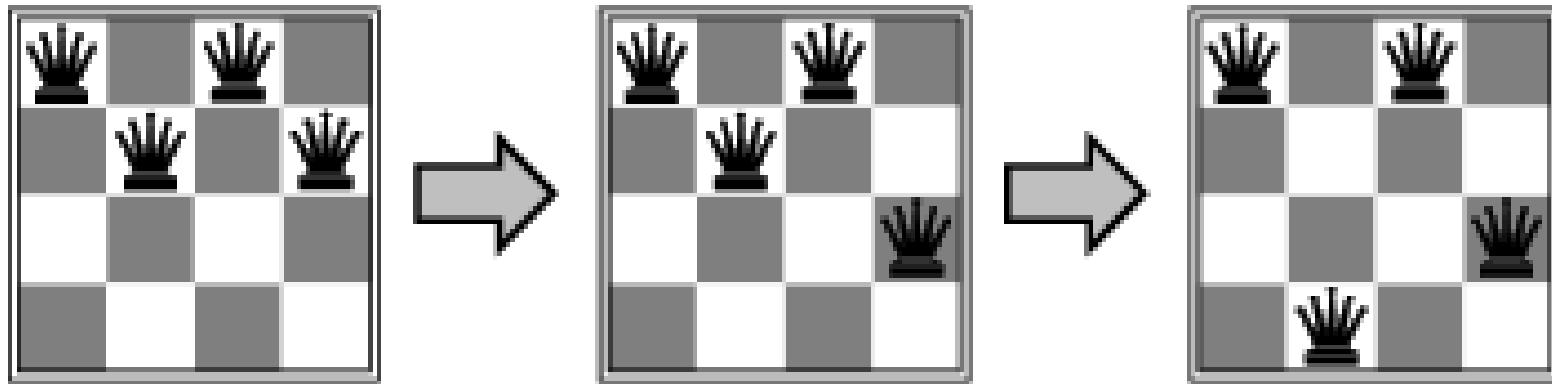
When elevation corresponds to **cost**, the aim is to find the **lowest valley – global minimum**.



- A complete local search algorithm always find a goal if one exists;
- An optimal algorithm always find a global minimum/maximum.

# Example: $n$ -queens

- Put  $n$  queens on an  $n \times n$  board with no two queens on the same row, column, or diagonal
- Local search: start with all  $n$ , move a queen to reduce conflicts



# Hill-climbing search

- "Like climbing Everest in thick fog with amnesia"

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

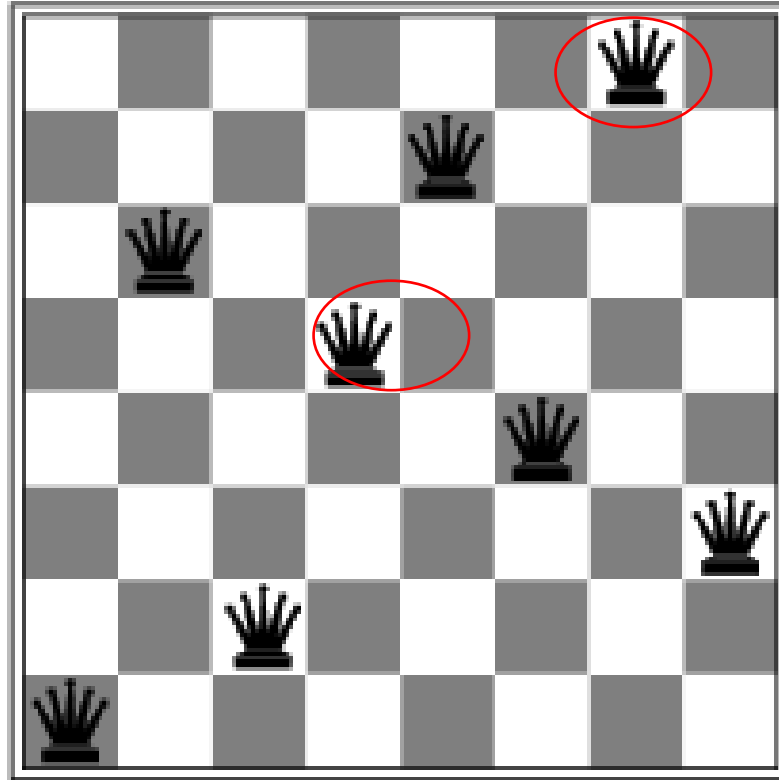
  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```

# Hill-climbing search: 8-queens problem

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 18 | 12 | 14 | 13 | 13 | 12 | 14 | 14 |
| 14 | 16 | 13 | 15 | 12 | 14 | 12 | 16 |
| 14 | 12 | 18 | 13 | 15 | 12 | 14 | 14 |
| 15 | 14 | 14 | ♚  | 13 | 16 | 13 | 16 |
| ♚  | 14 | 17 | 15 | ♚  | 14 | 16 | 16 |
| 17 | ♚  | 16 | 18 | 15 | ♚  | 15 | ♚  |
| 18 | 14 | ♚  | 15 | 15 | 14 | ♚  | 16 |
| 14 | 14 | 13 | 17 | 12 | 14 | 12 | 18 |

- $h$  = number of pairs of queens that are attacking each other, either directly or indirectly
- $h = 17$  for the above state

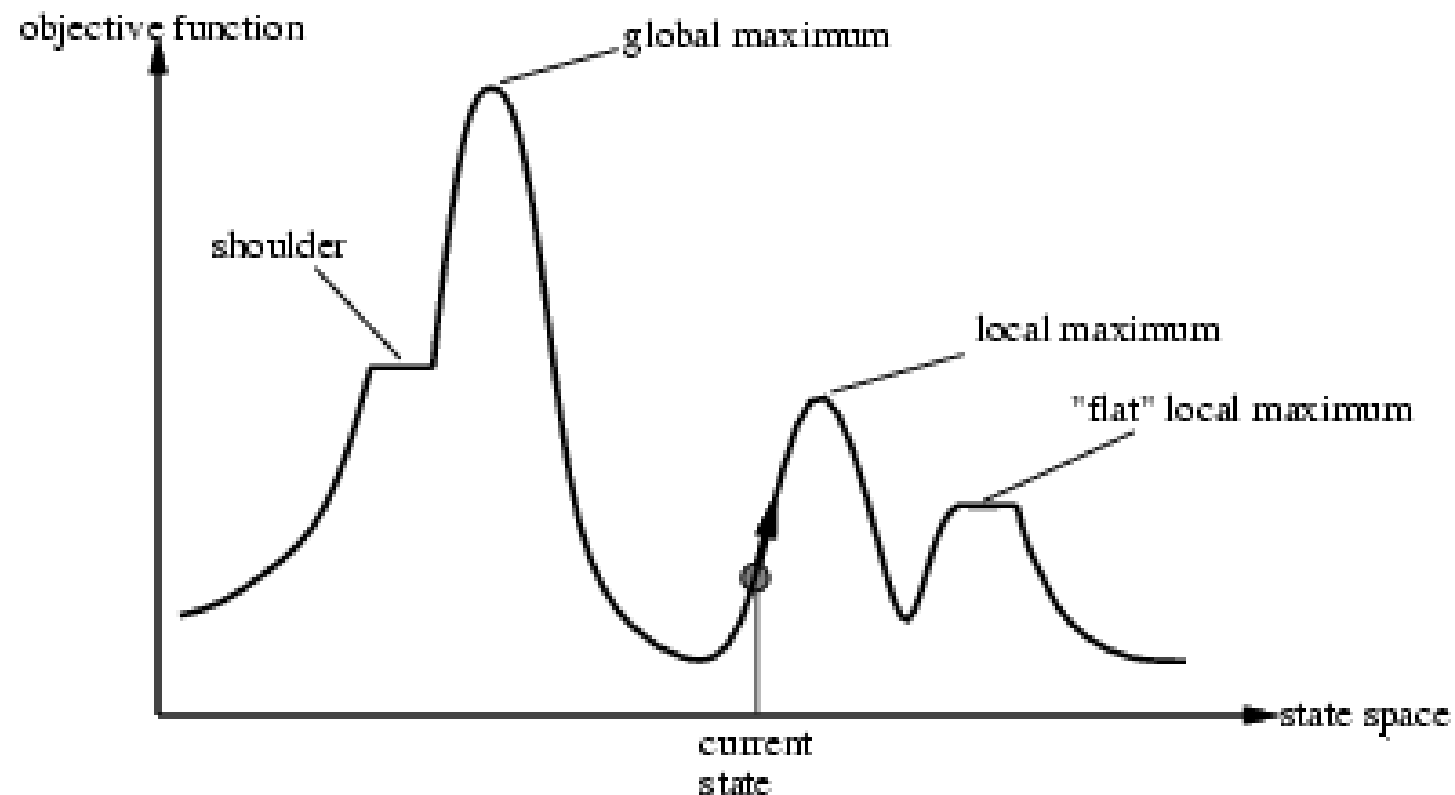
# Hill-climbing search: 8-queens problem



- A local minimum with  $h = 1$

# Hill-climbing search

- Problem: depending on initial state, can get stuck in local maxima



# Local Optima (Foothill Problem)

- **Problem Definition**

- Point reached by hill-climbing may be maximal but not maximum
- Maximal
  - Definition: not dominated by any neighboring point (with respect to criterion measure)
- Maximum
  - Definition: dominates all neighboring points (wrt criterion measure)

- **Ramifications**

- Steepest ascent hill-climbing will become trapped (why?)
- Need some way to break out of trap state
  - Accept transition (i.e., search move) to dominated neighbor
  - Start over: random restarts

# Lack of Gradient (Plateau Problem)

- **Problem Definition**

- Function space may contain points whose neighbors are indistinguishable (with respect to criterion measure)
- Effect: “flat” search landscape
- A hill-climber search might get lost on the plateau.

- **Ramifications**

- Steepest ascent hill-climbing will become trapped
- Need some way to break out of zero gradient
  - Accept transition (i.e., search move) to random neighbor
  - Random restarts
  - Take bigger steps

# Notes on Hill-Climber

- The success of the hill climber depends very much on the shape of the state-space landscape.
- If there are few local maxima and plateau, random-restart hill climbing will find a good solution very quickly.

# Simulated annealing search

- Idea: escape local maxima by allowing some "bad" moves but **gradually decrease** their frequency

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                   next, a node
                   T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

# Properties of simulated annealing search

- One can prove: If  $T$  decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1.
- Widely used in VLSI layout, airline scheduling, etc

<https://www.youtube.com/watch?v=FyyVbuLZav8>

## Steps for solving Sudoku

- 1- Generating random states
- 2- Writing a cost function
- 3- Selecting starting temperature
- 4- Calculating iterations per T
- 5- Choosing a cooling rate

|                   |   |   |   |   |   |   |   |   |   |
|-------------------|---|---|---|---|---|---|---|---|---|
| 1                 | 5 | 2 | 4 | 4 | 3 | 7 | 6 | 8 | 9 |
| 2                 | 6 | 9 | 7 | 1 | 5 | 9 | 2 | 7 | 3 |
| 2                 | 8 | 1 | 3 | 6 | 8 | 2 | 4 | 1 | 5 |
| 1                 | 4 | 3 | 1 | 6 | 7 | 5 | 4 | 8 | 1 |
| 2                 | 5 | 8 | 6 | 1 | 8 | 9 | 9 | 3 | 2 |
| 2                 | 7 | 9 | 2 | 2 | 3 | 4 | 7 | 6 | 5 |
| 2                 | 2 | 8 | 9 | 7 | 1 | 5 | 8 | 4 | 2 |
| 2                 | 6 | 4 | 5 | 2 | 9 | 3 | 9 | 1 | 3 |
| 2                 | 3 | 1 | 7 | 8 | 6 | 4 | 7 | 5 | 6 |
| 2 3 1 3 2 3 3 2 3 |   |   |   |   |   |   |   |   |   |

Total cost = 38

# Tabu Search

- Tabu: prevent returning quickly to same state.
- Implementation: Keep fixed length queue (“tabu list”): add most recent step to queue; drop “oldest” step.
- Never make step that's currently on the tabu list.
- Quite powerful; competitive with simulated annealing

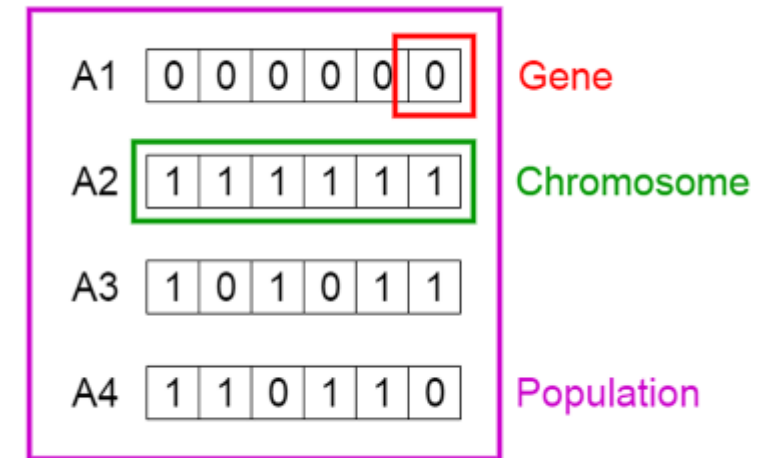
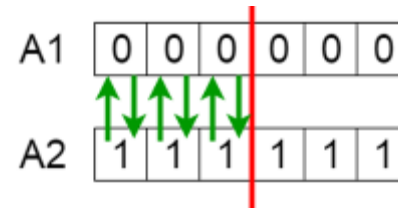
# Local beam search

- Keep track of  $k$  states rather than just one
- Start with  $k$  randomly generated states
- At each iteration, all the successors of all  $k$  states are generated
- If any one is a goal state, stop; else select the  $k$  best successors from the complete list and repeat.

# Local beam search

- Useful information is passed among parallel search threads.
- The states generating the best successors says to others “Come over here, the grass is greener” 😊
- However, it can suffer from a lack of diversity among the  $k$  states.
  - They can quickly become concentrated in a small region of the state space.
  - Stochastic local beam search helps alleviate this problem.
    - “Choosing  $k$  successors at random with the probability of choosing a given successor being an increasing function of its value”

# Genetic algorithms



- A successor state is generated by combining two parent states.
- Start with  $k$  randomly generated states (**population**)
- A state is represented as a string over a finite alphabet (often a string of 0s and 1s)
- Evaluation function (**fitness function**). Higher values for better states.
- Produce the next generation of states by selection, crossover, and mutation.

# Phases of Genetic Algorithm

## 1. Initial population:

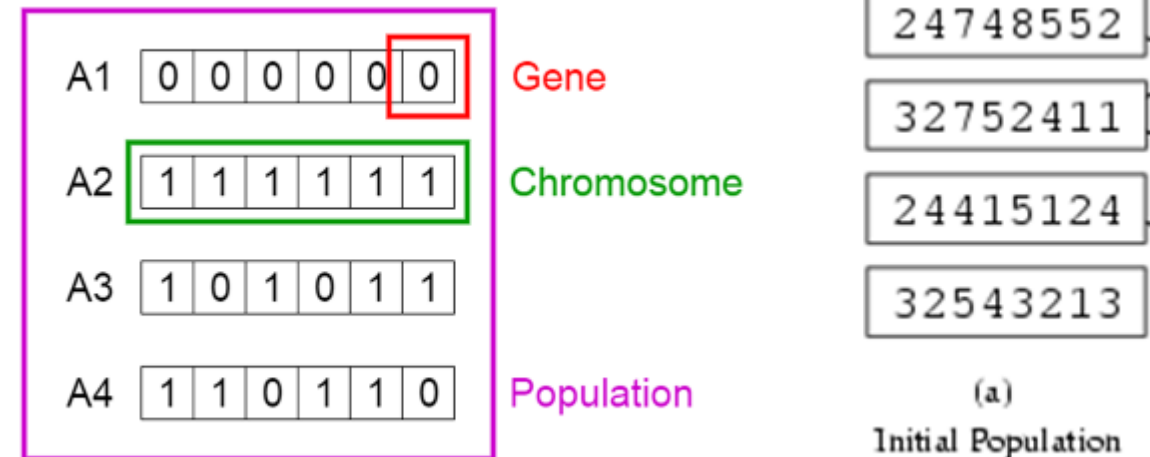
- “The process begins with a set of individuals which is called a **Population**.
- Each individual (**Chromosome**) is a solution to the problem you want to solve.
- In a genetic algorithm, the set of genes of an individual is represented using a string, in terms of an alphabet. Usually, binary values are used (string of 1s and 0s).”

## 2. Fitness function

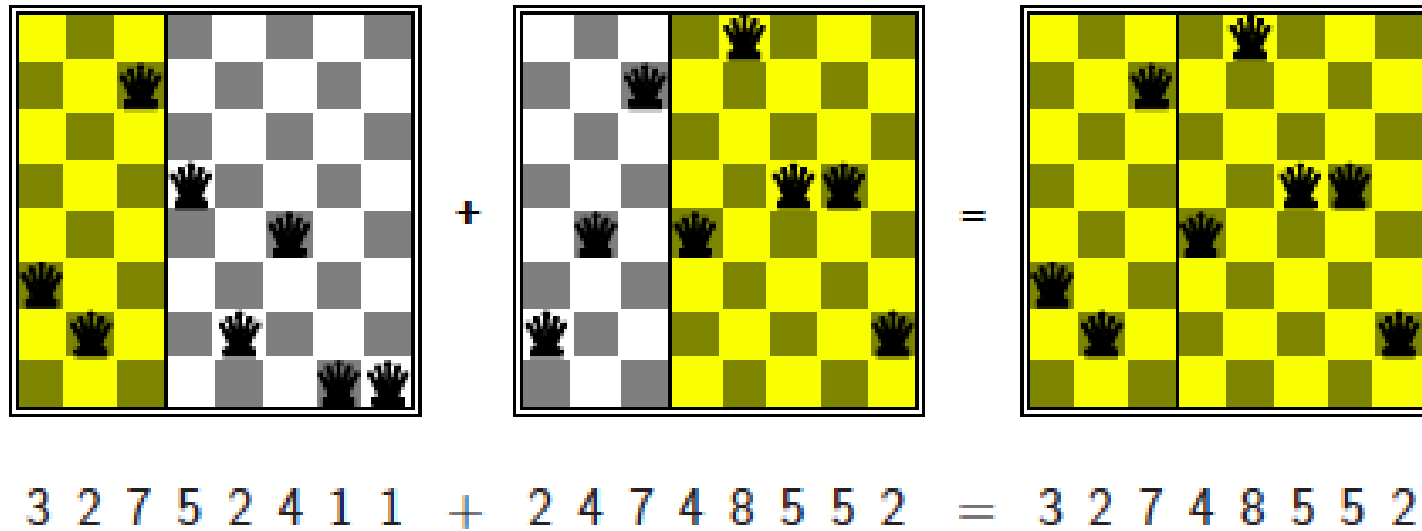
## 3. Selection

## 4. Crossover

## 5. Mutation



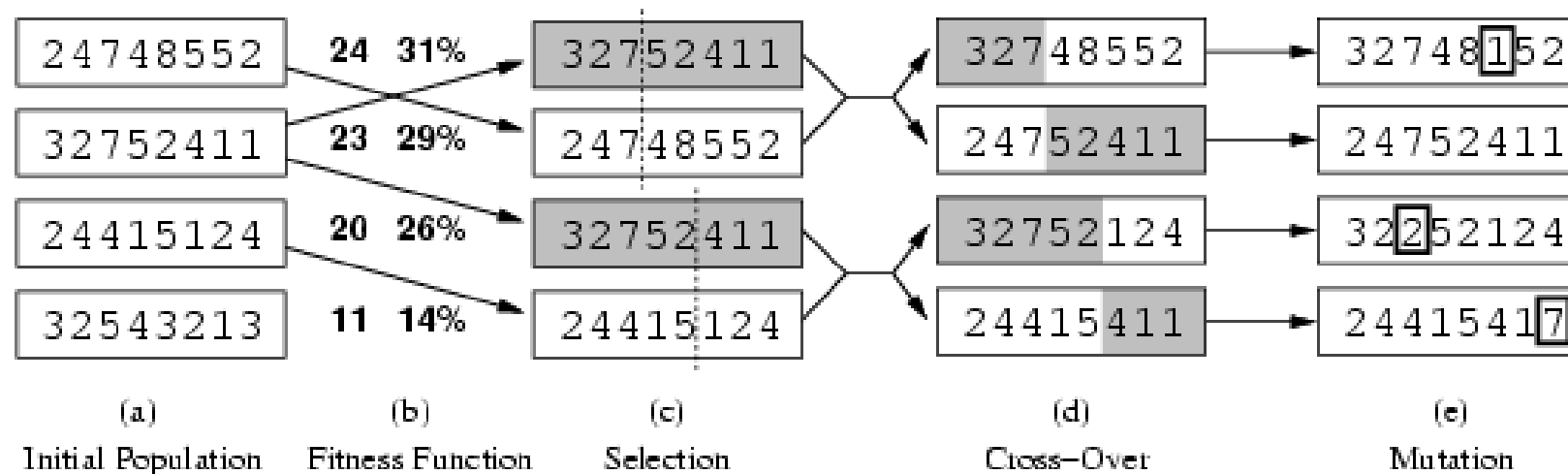
# Genetic algorithms



## Phase 2 :Fitness Function

- Each individual (state) is evaluated/rated by the objective function – called fitness function.
- For example, for the 8 queens problem we can use the number of non-attacking pairs of the queens – which is 28 for the optimal solution.
- The probability that an individual will be selected for reproduction is based on its fitness score.

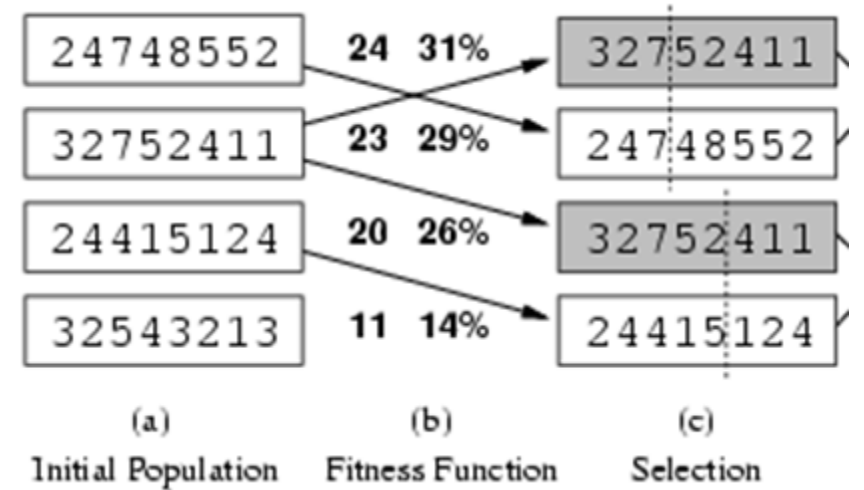
# Genetic algorithms



- Fitness function: number of non-attacking pairs of queens (min = 0, max =  $8 \times 7/2 = 28$ )
- $24/(24+23+20+11) = 31\%$
- $23/(24+23+20+11) = 29\%$  etc

# Phase 3: Selection

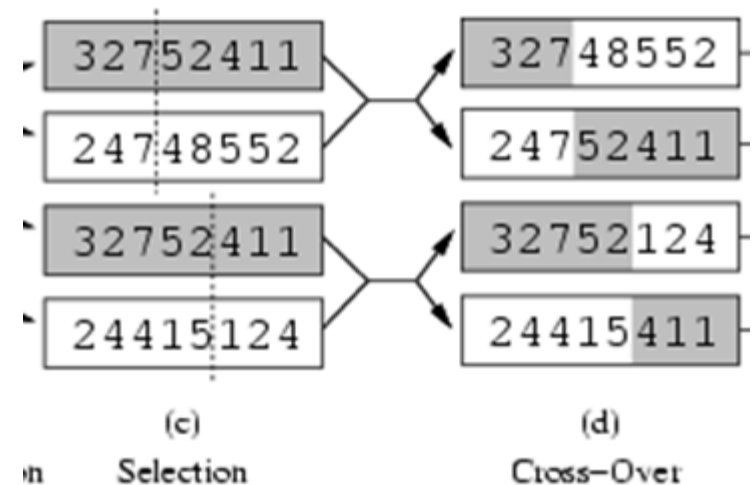
- Two pairs are selected randomly for reproduction, in accordance with the probabilities.
- Individuals with high fitness have more chance to be selected for reproduction.



Many variants of this selection rule. The one used above is called “culling method” in which the individuals below a given threshold are discarded, which converge faster than the random version.

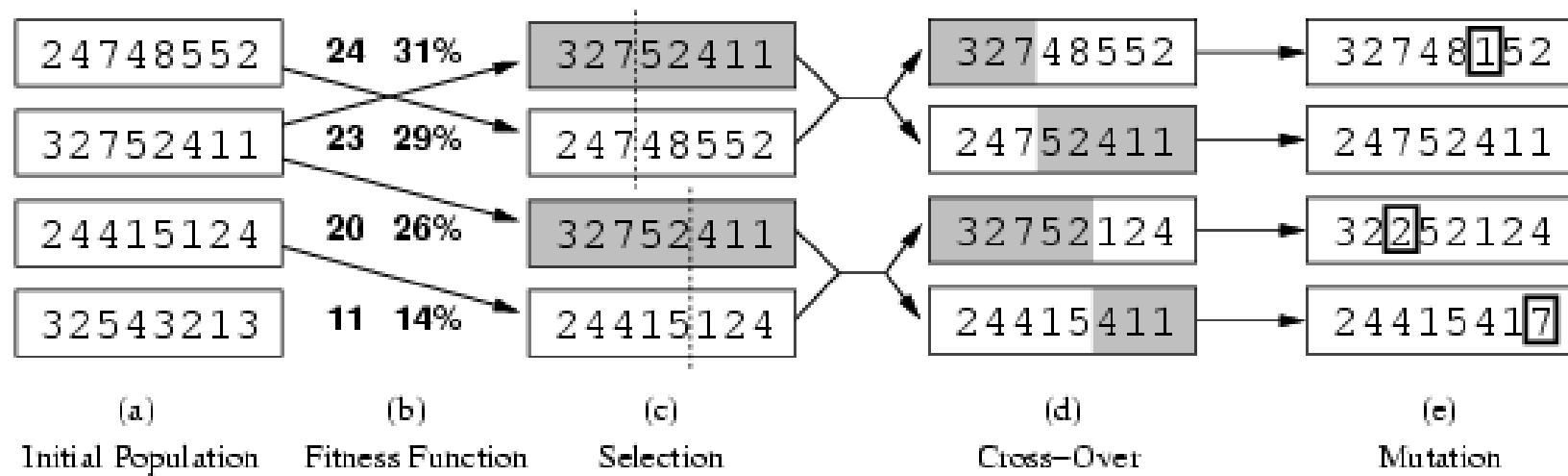
# Phase 4: Crossover

- For each pair to be mated, a crossover point is chosen randomly from the positions in the string.
- **Offspring** are created by crossing over the Parent strings at the cross-over point.



# Phase 5: Mutation

- Each location is subject to random mutation with a small independent probability.



# Genetic Algorithm Example:

<https://www.youtube.com/watch?v=uQj5UNhCPuo>

